

Python Complete Revision Guide

Master Every Core Concept

Variables • Data Types • Operators • Conditions • Loops • Functions • Args/Kwargs • Scope • Decorators • Closures

CHAPTER 1 — Variables & Data Types

What is a Variable?

A variable is a named container that stores a value in memory. Python is dynamically typed — you don't declare types, Python figures it out automatically.

```
# Creating variables — no type declaration needed
name = "Arjun"          # Python knows this is a string
age = 20                 # Python knows this is an integer
height = 5.9            # Python knows this is a float
is_student = True       # Python knows this is a boolean

# Check the type anytime
print(type(name))       # <class str>
print(type(age))        # <class int>
```

All Data Types — Quick Reference

Type	Example	Use Case
int	42, -10, 0	Counting, indexing
float	3.14, -0.5	Decimals, measurements
str	"hello", 'world'	Text, names, messages
bool	True, False	Conditions, flags
list	[1, 2, 3]	Ordered, changeable collection
tuple	(1, 2, 3)	Ordered, unchangeable collection
dict	{'a': 1}	Key-value pairs
set	{1, 2, 3}	Unique items, no duplicates
NoneType	None	Empty/missing value

Strings — Deep Dive

```
# String creation
single = 'hello'
double = "world"
multi = """This is
a multiline string"""

# String operations
name = "arjun"
print(name.upper())      # ARJUN
```

```

print(name.capitalize())    # Arjun
print(name.replace("a","@")) # @rjun
print(len(name))            # 5

# f-strings (use these always)
age = 20
print(f"My name is {name}, I am {age} years old")
print(f"Next year I will be {age + 1}")
print(f"Pi is {3.14159:.2f}") # Pi is 3.14

# String indexing
word = "Python"
print(word[0])    # P      (first)
print(word[-1])   # n      (last)
print(word[0:3])  # Pyt    (slice)
print(word[::-1]) # nohtyP (reverse)

```

| Lists — Deep Dive

```

fruits = ["apple", "banana", "mango"]

# Access
print(fruits[0])    # apple
print(fruits[-1])   # mango

# Modify
fruits.append("grape")    # add to end
fruits.insert(1, "kiwi")  # add at index 1
fruits.remove("banana")   # remove by value
fruits.pop()              # remove last item
fruits.pop(0)             # remove by index

# Slicing
nums = [0, 1, 2, 3, 4, 5]
print(nums[1:4])          # [1, 2, 3]
print(nums[::2])           # [0, 2, 4] every 2nd
print(nums[::-1])          # [5, 4, 3, 2, 1, 0] reversed

```

| Dictionaries — Deep Dive

```

student = {
    "name": "Arjun",
    "age": 20,
    "grade": "A"
}

# Access
print(student["name"])    # Arjun
print(student.get("age"))  # 20
print(student.get("marks", 0)) # 0 (default if missing)

# Modify
student["age"] = 21        # update

```

```
student["city"] = "Vadodara"    # add new key
del student["grade"]           # delete key

# Loop through
for key, value in student.items():
    print(f"{key}: {value}")

# Useful methods
print(student.keys())           # all keys
print(student.values())         # all values
```

Strings and Tuples are IMMUTABLE — you cannot change them after creation. Lists and Dicts are MUTABLE — you can change them.

`student["missing_key"]` raises `KeyError`. Always use `student.get("key")` for safe access.

CHAPTER 2 — Operators

All Operator Types

Operator Type	Symbols	Example
Arithmetic	+ - * / // % **	10 // 3 = 3, 2**8 = 256
Comparison	== != > < >= <=	5 > 3 → True
Logical	and or not	True and False → False
Assignment	= += -= *= /=	x += 5 means x = x + 5
Identity	is, is not	x is None
Membership	in, not in	"a" in "apple" → True
Bitwise	& ^ ~ << >>	used in low-level operations

The Tricky Operators

```
# // Floor division — always rounds DOWN
print(10 // 3)    # 3 (not 3.33)
print(-7 // 2)    # -4 (rounds DOWN, not toward zero!)

# ** Power
print(2 ** 10)    # 1024
print(9 ** 0.5)   # 3.0 (square root)

# % Modulo — gives REMAINDER
print(10 % 3)     # 1
print(15 % 5)     # 0 (divisible)
# Trick: even/odd check
print(7 % 2)      # 1 → odd
print(8 % 2)      # 0 → even

# == vs is
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)     # True (same VALUE)
print(a is b)     # False (different OBJECT in memory)

x = None
print(x is None)  # True — always use "is" for None check

# Logical operators
print(True and False) # False (both must be True)
print(True or False)  # True (one must be True)
print(not True)        # False

# Short circuit evaluation
x = 0
result = x and (10/x)  # Safe! x is False so (10/x) never runs
```

```
# Ternary operator
age = 20
status = "adult" if age >= 18 else "minor"
print(status)    # adult
```

`==` checks VALUE equality. `is` checks if they are the SAME object in memory. Never use `is` to compare numbers or strings — use `==` for that.

For `None`, always write: `if x is None` — NOT `if x == None`

CHAPTER 3 — Conditions

| if / elif / else — The Full Structure

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(f"Grade: {grade}")    # Grade: B
```

| Truthy & Falsy — Critical Concept

In Python, everything has a boolean value. These are ALL considered False:

```
# FALSEY values – all treated as False in conditions
False
None
0
0.0
""          # empty string
[]          # empty list
{}          # empty dict
()          # empty tuple

# TRUTHY – everything else is True
"hello"     # non-empty string → True
[1,2,3]     # non-empty list → True
42          # non-zero → True

# Practical use
name = ""
if name:    # empty string is Falsy
    print("Hello", name)
else:
    print("No name given") # this runs

numbers = [1, 2, 3]
if numbers: # non-empty list is Truthy
    print("List has items") # this runs
```

| Nested Conditions & Combining

```
# Combining conditions
age = 25
has_id = True

if age >= 18 and has_id:
    print("Entry allowed")

# "in" operator for membership
fruits = ["apple", "mango", "banana"]
if "mango" in fruits:
    print("Mango found!")

# Chained comparison (Python unique feature)
x = 15
if 10 < x < 20:                # instead of x > 10 and x < 20
    print("x is between 10 and 20")
```

Python allows chained comparisons like $10 < x < 20$. Very clean and readable.

Remember: a single `=` is assignment. `==` is comparison. Common beginner bug!

CHAPTER 4 — Loops

for Loop — Iterate Over Anything

```
# Loop over list
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)

# Loop over string
for char in "Python":
    print(char)    # prints P y t h o n

# Loop with range
for i in range(5):    # 0 1 2 3 4
    print(i)

for i in range(2, 8):    # 2 3 4 5 6 7
    print(i)

for i in range(0, 10, 2):    # 0 2 4 6 8 (step=2)
    print(i)

# enumerate — index + value together
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")    # 0: apple, 1: banana...

# zip — loop two lists together
names = ["Arjun", "Riya", "Sam"]
scores = [90, 85, 92]
for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

while Loop — Run Until Condition is False

```
count = 0
while count < 5:
    print(count)
    count += 1    # MUST update — or infinite loop!

# while with break
while True:
    user_input = input("Type quit to exit: ")
    if user_input == "quit":
        break    # exit loop
    print(f"You typed: {user_input}")
```

break, continue, pass — Loop Control

```

# break – exit loop completely
for i in range(10):
    if i == 5:
        break          # stops at 5
    print(i)           # prints 0 1 2 3 4

# continue – skip current iteration
for i in range(10):
    if i % 2 == 0:
        continue      # skip even numbers
    print(i)           # prints 1 3 5 7 9

# pass – do nothing (placeholder)
for i in range(5):
    pass              # loop runs but does nothing

# else on loop – runs when loop finishes normally (no break)
for i in range(5):
    print(i)
else:
    print("Loop done!") # runs after loop finishes

```

Forgetting to update the counter in a while loop causes an INFINITE loop. Always make sure the condition will eventually become False.

enumerate() and zip() are used constantly in real Python code. Master them!

CHAPTER 5 — Functions

Why Functions?

Functions let you write code ONCE and reuse it many times. They also make your code organized, readable, and testable.

```
# Basic function
def greet(name):
    return f"Hello, {name}!"

print(greet("Arjun"))    # Hello, Arjun!
print(greet("Python"))   # Hello, Python!

# Function with default parameter
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Arjun"))      # Hello, Arjun!
print(greet("Arjun", "Namaste")) # Namaste, Arjun!

# Multiple return values
def min_max(numbers):
    return min(numbers), max(numbers)    # returns tuple

low, high = min_max([3, 1, 8, 2, 9])
print(low, high)    # 1 9
```

Return vs Print — Critical Difference

```
# print() — just displays, gives nothing back
def add_print(a, b):
    print(a + b)

result = add_print(3, 5)    # shows 8
print(result)               # None ← got nothing back!

# return — gives value back to caller
def add_return(a, b):
    return a + b

result = add_return(3, 5)   # no output yet
print(result)               # 8 ← we have the value!

# Use return when you need the value for something else
total = add_return(3, 5) * 2    # 16 — works!
total = add_print(3, 5) * 2     # Error! None * 2
```

Lambda — Anonymous One-Line Functions

```
# Regular function
def square(x):
    return x * x

# Same as lambda
square = lambda x: x * x
print(square(5))    # 25

# Lambda with multiple args
add = lambda a, b: a + b
print(add(3, 5))    # 8

# Real use — sorting with custom key
students = [("Arjun", 85), ("Riya", 92), ("Sam", 78)]
students.sort(key=lambda s: s[1])    # sort by score
print(students)    # [("Sam",78), ("Arjun",85), ("Riya",92)]

# With map() and filter()
nums = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, nums))    # [1,4,9,16,25]
evens = list(filter(lambda x: x%2==0, nums)) # [2, 4]
```

Use lambda for short, simple operations. For anything complex (more than one line), use a regular def function.

return exits the function immediately. Any code after return in the same block never runs.

CHAPTER 6 — *args & **kwargs

The Problem They Solve

What if you don't know how many arguments a function will receive? *args and **kwargs handle unlimited inputs.

*args — Unlimited Positional Arguments

```
# Without *args — limited to fixed number
def add(a, b):
    return a + b          # only 2 numbers, what about 5?

# With *args — unlimited
def add(*args):
    print(type(args))    # <class tuple>
    return sum(args)

print(add(1, 2))          # 3
print(add(1, 2, 3, 4, 5)) # 15
print(add(10, 20, 30))   # 60

# args is just a TUPLE inside the function
def show(*args):
    for i, val in enumerate(args):
        print(f"arg{i}: {val}")

show("python", 42, True)
# arg0: python
# arg1: 42
# arg2: True
```

**kwargs — Unlimited Keyword Arguments

```
# **kwargs gives you a DICTIONARY of named arguments
def profile(**kwargs):
    print(type(kwargs))    # <class dict>
    for key, value in kwargs.items():
        print(f"{key}: {value}")

profile(name="Arjun", age=20, city="Vadodara")
# name: Arjun
# age: 20
# city: Vadodara
```

Combining All Together — The Full Order

```

# Order MUST be: normal, *args, keyword defaults, **kwargs
def full_function(name, *args, greeting="Hello", **kwargs):
    print(f"{greeting}, {name}!")
    print(f"Extra args: {args}")
    print(f"Extra kwargs: {kwargs}")

full_function("Arjun", 1, 2, 3, greeting="Namaste", city="Vadodara")
# Namaste, Arjun!
# Extra args: (1, 2, 3)
# Extra kwargs: {city: Vadodara}

# Unpacking with * and **
nums = [1, 2, 3]
def add(a, b, c):
    return a + b + c

print(add(*nums))          # unpacks list → add(1, 2, 3)

info = {"a": 1, "b": 2, "c": 3}
print(add(**info))         # unpacks dict → add(a=1, b=2, c=3)

```

The names "args" and "kwargs" are just conventions. You could write `*numbers` or `**options` — the `*` and `**` are what matter.

Order matters! `def f(*args, **kwargs)` works. `def f(**kwargs, *args)` raises `SyntaxError`.

CHAPTER 7 — Scope (LEGB Rule)

The LEGB Rule — Python's Search Order

When Python sees a variable name, it searches in this exact order:

Level	Stands For	Where
L	Local	Inside the current function
E	Enclosing	Inside outer function (for nested functions)
G	Global	Top level of the file
B	Built-in	Python's own names (print, len, range...)

```
x = "global"           # G – Global

def outer():
    x = "enclosing"     # E – Enclosing

    def inner():
        x = "local"     # L – Local
        print(x)        # finds L first → "local"

    inner()
    print(x)            # finds E → "enclosing"

outer()
print(x)                # finds G → "global"
```

global Keyword — Modify Global from Inside Function

```
count = 0               # global variable

def increment():
    global count         # tell Python: use the GLOBAL count
    count += 1

increment()
increment()
print(count)            # 2

# Without global keyword – creates a NEW local variable
def bad_increment():
    count += 1           # UnboundLocalError! Python sees a local count
                        # that was never assigned
```

nonlocal Keyword — Modify Enclosing Variable

```
def make_counter():
    count = 0

    def increment():
        nonlocal count    # use ENCLOSING count, not local
        count += 1
        return count

    return increment

c = make_counter()
print(c())    # 1
print(c())    # 2
print(c())    # 3
```

Avoid global variables whenever possible. They make code hard to debug. Use function parameters and return values instead.

If a function has ANY assignment to a variable name, Python treats it as LOCAL throughout that function — even before the assignment line. This causes `UnboundLocalError`.

CHAPTER 8 — Decorators

What is a Decorator?

A decorator is a function that takes another function, adds behavior to it, and returns a new improved version. Write extra logic ONCE, apply it to any function.

Basic Structure — Step by Step

```
# Step 1: A function that takes a function
def my_decorator(func):

    # Step 2: Wrapper adds behavior
    def wrapper(*args, **kwargs):
        print("Before the function")      # extra behavior
        result = func(*args, **kwargs)   # run original
        print("After the function")      # extra behavior
        return result                    # must return!

    return wrapper    # return wrapper, NOT wrapper()

# Step 3: Apply with @ syntax
@my_decorator
def say_hello(name):
    print(f"Hello {name}!")

say_hello("Arjun")
# Before the function
# Hello Arjun!
# After the function

# @ is just shorthand for:
# say_hello = my_decorator(say_hello)
```

Real-World Decorators

```
import time
from functools import wraps

# Timer decorator
def timer(func):
    @wraps(func)    # preserves original function name
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {(end-start)*1000:.2f}ms")
        return result
    return wrapper
```

```

# Logger decorator
def logger(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with {args} {kwargs}")
        result = func(*args, **kwargs)
        print(f"Returned: {result}")
        return result
    return wrapper

# Validator decorator
def validate_positive(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        for arg in args:
            if isinstance(arg, (int, float)) and arg < 0:
                raise ValueError(f"Negative not allowed: {arg}")
        return func(*args, **kwargs)
    return wrapper

```

Decorator With Arguments — 3-Layer Pattern

```

def repeat(n):
    # Layer 1: takes the argument
    def decorator(func): # Layer 2: takes the function
        # Layer 3: runs on call
        def wrapper(*args, **kwargs):
            for i in range(n):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def say(msg):
    print(msg)

say("Hello!") # prints Hello! three times

```

Forgetting return result means your decorated function always returns None — silent bug!

Writing return wrapper() instead of return wrapper calls the function immediately instead of returning it.

Always use @wraps(func) from functools — it preserves the original function's name and docstring.

CHAPTER 9 — Closures

What is a Closure?

A closure is a function that remembers variables from its enclosing scope even after that scope has finished executing. Like a function with a built-in memory.

3 Conditions for a Closure

Condition	Description
1. Nested function	A function defined inside another function
2. Variable used	Inner function uses a variable from outer function
3. Returned	Outer function returns the inner function

```
def outer():
    msg = "Hello"          # outer variable

    def inner():
        print(msg)         # inner USES outer variable

    return inner           # outer RETURNS inner

my_func = outer()         # outer() finished – msg should be gone
my_func()                 # but it prints "Hello" – msg is ALIVE!

# Proof: see the saved variables
print(my_func.__closure__[0].cell_contents)  # Hello
```

nonlocal in Closures

```
def make_counter():
    count = 0

    def increment():
        nonlocal count    # modify outer variable
        count += 1
        return count

    return increment

c1 = make_counter()      # independent counter 1
c2 = make_counter()      # independent counter 2

c1()    # 1
c1()    # 2
c1()    # 3
```

```
c2()    # 1 ← c2 has its OWN separate count
```

Function Factory — Practical Closure Use

```
# Generate custom multiplier functions
def multiplier(factor):
    def multiply(number):
        return number * factor    # factor remembered from outer
    return multiply

double = multiplier(2)
triple = multiplier(3)

print(double(5))    # 10
print(triple(5))    # 15
print(double(10))   # 20
```

Classic Closure Trap — Loop Bug

```
# BROKEN — all functions share same variable
funcs = []
for i in range(3):
    def f():
        print(i)
    funcs.append(f)

funcs[0]()    # expects 0, prints 2
funcs[1]()    # expects 1, prints 2
funcs[2]()    # expects 2, prints 2

# FIXED — capture value at creation time
funcs = []
for i in range(3):
    def f(x=i):    # x=i captures current value of i
        print(x)
    funcs.append(f)

funcs[0]()    # 0
funcs[1]()    # 1
funcs[2]()    # 2
```

Decorators are just closures. The wrapper function closes over the func variable from the decorator. Now you know WHY decorators work.

All functions in a loop share the same loop variable. Use default argument `x=i` to capture the value at that moment.

Quick Reference — All Chapters

Concept	One-Line Summary
Variables	Named containers. Python auto-detects type.
Data Types	int, float, str, bool, list, dict, tuple, set, None
Operators	// floor div, % remainder, ** power, is identity, in membership
Conditions	if/elif/else. Truthy/Falsy. Ternary: x if cond else y
Loops	for iterates collections. while runs until False. break/continue control
Functions	def keyword. return gives value back. lambda for one-liners
*args/**kwargs	*args = tuple of extras. **kwargs = dict of named extras
Scope (LEGB)	Local → Enclosing → Global → Built-in. global/nonlocal to modify
Decorators	@syntax. Function that wraps another. Add behavior without editing
Closures	Inner function remembers outer variable. Basis of decorators